

Rynek Java w Polsce i plan wyjścia z legacy do nowoczesnego backendu

Co rynek mówi o twojej sytuacji

Najważniejszy wniosek po przejrzeniu aktualnych ofert, raportów Just Join IT i No Fluff Jobs, konkretnych ogłoszeń dla osób z około 1–2 latami doświadczenia oraz materiałów o współczesnych rekrutacjach jest prosty:

nie masz problemu pod tytułem „mam za mało doświadczenia”. Masz problem pod tytułem „moje doświadczenie jest słabo dopasowane do stacku, którego oczekuje współczesny rynek”.

To jest zdecydowanie łatwiejszy problem do naprawienia.

Masz już coś, czego nie ma osoba po kursie: około 1,5 roku rzeczywistej pracy przy komercyjnym systemie, wcześniejszy staż, pracę z istniejącym kodem, poprawki, wymagania biznesowe, legacy, debugging i prawdopodobnie sytuacje, w których rozwiązanie trzeba dowieźć mimo bałaganu. Nie powinieneś więc przygotowywać się jak kandydat na pierwszą pracę. Powinieneś przygotowywać się jak **Java Developer z doświadczeniem, który musi uzupełnić modern-backend gap.**

Rynek jednocześnie nie jest szczególnie przyjazny dla czystych juniorów. Według raportu Just Join IT za 2025 r. tylko **4,79% ofert było oznaczonych jako junior**, podczas gdy **43,73% dotyczyło mida**, a 51,48% seniora. Java odpowiadała za około **9,46% wszystkich ofert**, pozostając jedną z największych kategorii technologicznych na portalu. ¹ No Fluff Jobs podaje jednocześnie, że Backend nadal należy do kategorii o największej liczbie ofert; średnia liczba aplikacji na ofertę na ich platformie spadła z 44 w 2024 r. do 24 w 2025 r. i 21 w pierwszej połowie 2026 r., chociaż juniorzy nadal mają trudniejszą sytuację niż osoby bardziej doświadczone. ²

To jest dla ciebie ważne, bo **nie powinieneś celować wyłącznie w ogłoszenia z napisem Junior.**

Aktualne oferty pokazują dużą rozbieżność w nazewnictwie seniority. Aviva miała ofertę oznaczoną jako Mid, w której oczekiwano „around 2 years” doświadczenia z Javą i pewnego doświadczenia ze Spring Bootem. ³ Z drugiej strony Xebia oznacza aktualną ofertę jako Mid, ale wymaga w niej ponad czterech lat doświadczenia. ⁴ SmartBear ma ofertę Junior Backend Engineer kierowaną do osób z **1–2+ latami** doświadczenia w Javie. ⁵ Jacobs w ogłoszeniu „Junior/Mid Java Developer” oczekuje minimum dwóch lat praktycznej Javy. ⁶

Wniosek: kiedy będziesz miał 1,5 roku pracy + 3 miesiące stażu, aplikuj równolegle na:

- Junior Java Developer,
- Junior Backend Engineer,
- Junior+/Strong Junior,
- Java Developer bez seniority w tytule,
- Software Engineer I / Associate Software Engineer,
- Regular Java Developer,

- wybrane Mid Java Developer, jeżeli wymagają około 2–3 lat, a nie 4–5+.

Nie czekaj, aż spełnisz „2 years” co do miesiąca.

Największym zagrożeniem w twoim CV będzie natomiast pytanie:

„Masz prawie dwa lata doświadczenia w Javie. To dlaczego praktycznie nie pracowałeś ze Spring Bootem?”

Musisz doprowadzić do sytuacji, w której odpowiedź brzmi mniej więcej:

„Pracowałem komercyjnie przy dużym legacy systemie Java/Java EE, więc mam doświadczenie z rzeczywistym kodem produkcyjnym i utrzymaniem systemu. Jednocześnie świadomie uzupełniłem nowoczesny stack backendowy: Spring Boot, REST, JPA/Hibernate, PostgreSQL, testy integracyjne, Docker, CI/CD i podstawy messagingu. Zbudowałem na tym kompletną aplikację i potrafię wyjaśnić, jak te elementy działają.”

A nie:

„Niestety firma używała starego gówna i robiłem wszystko z AI.”

Drugiej wersji **nigdy nie mów na rozmowie**, nawet jeżeli jest prawdziwa.

Mapa umiejętności wynikająca z ofert

Nie podaję fałszywych procentów typu „Spring występuje w 83,7% ofert”, bo publiczne job boardy nie udostępniają mi czystego, zdeduplikowanego zbioru wszystkich ofert, a część ofert występuje na kilku portalach równocześnie. Zamiast tego używam skali: **powszechne / bardzo częste / częste / selektywne** i opieram ją na powtarzalności wymagań w aktualnych ogłoszeniach oraz raportach rynku.

Wśród aktualnych ofert szczególnie wymowne są m.in. SmartBear, który przy 1–2+ latach doświadczenia wymienia Java 21+, Spring Boot 3+, REST, RDBMS, OOP, design patterns i korzystanie z AI; BAE Systems, które dla Junior Java Developer wymaga do dwóch lat doświadczenia, co najmniej roku Spring Boot/microservices, testów, Git, CI i Mavena; Jacobs, który dla Junior/Mid oczekuje Spring/Spring Boot, Git, SQL, JUnit, Linuxa i znajomości Dockera/Kubernetesa; oraz Rockitworks, które przy dwóch latach wymienia Spring Boot, Spring Security, JWT, testy Springa, JUnit i relacyjną bazę SQL. ⁷

Obszar	Rynek dla ~1,5–2 lat	Priorytet dla ciebie	Poziom, do którego masz dojść
Core Java / OOP	Powszechne	● Krytyczny	Swobodnie, bez AI
Collections / generics / equals/hashCode	Często sprawdzane jako fundament Javy	● Krytyczny	Umieć wyjaśnić i kodować
Streams / lambda / Optional	Bardzo częste	● Krytyczny	Codzienne użycie
Java 17/21 i nowsza Java	Bardzo częste w modern backend	● Krytyczny	Praktycznie 17/21, świadomość 25

Obszar	Rynek dla ~1,5–2 lat	Priorytet dla ciebie	Poziom, do którego masz dojść
Spring Core / Boot	Powszechne	 Krytyczny	Samodzielna aplikacja
Spring MVC / REST	Powszechne	 Krytyczny	Samodzielnie
Spring Data JPA / Hibernate	Bardzo częste	 Krytyczny	Samodzielnie + pułapki ORM
SQL / PostgreSQL/ MySQL	Bardzo częste	 Krytyczny	Swobodne zapytania
JUnit / Mockito	Bardzo częste	 Krytyczny	Samodzielnie
Testy integracyjne	Częste	 Bardzo ważny	Praktycznie
Git	Powszechne	 Krytyczny	Codzienne operacje
Maven	Bardzo częste	 Krytyczny	Build/dependencies/ profiles
Gradle	Częste, ale mniej kluczowe	 Średni	Czytanie i podstawy
Docker	Częste	 Bardzo ważny	Dockerfile + Compose
CI/CD	Częste	 Bardzo ważny	Rozumieć pipeline
Linux	Częste	 Bardzo ważny	Podstawowa praca terminalowa
Microservices	Częste	 Bardzo ważny	Koncepcje, trade-offy
Kafka	Powtarzalne, ale nie uniwersalne	 Ważny	Producer/consumer + fundamentals
RabbitMQ	Selektywne	 Średni	Koncepcje
Kubernetes	Częste w stacku, rzadziej core juniora	 Średni	Podstawy
AWS/Azure/GCP	Częste, ale często dodatkowe	 Średni	Podstawy jednego cloud
Spring Security/JWT	Powtarzalne	 Ważny	Auth/authz + JWT
OAuth2/OIDC	Selektywne na twoim poziomie	 Średni	Koncepcje
JVM / pamięć / GC	Nie zawsze w ogłoszeniu, wartościowe na interview	 Ważny	Dobre podstawy
Concurrency	Zależne od roli	 Ważny	Fundamentals
Redis/cache	Selektywne	 Średni	Fundamentals

Obszar	Rynek dla ~1,5–2 lat	Priorytet dla ciebie	Poziom, do którego masz dojść
React/Angular	Głównie fullstack	 / 	Nie kosztem backendu
Testcontainers	Biblioteka nie jest uniwersalnym wymaganiem		Użyć w projekcie
WireMock	Selektywne		Umieć użyć
REST Assured	Selektywne		Opcjonalne
Clean Architecture/DDD	Częściej wyżej niż Junior		Rozumieć, nie studiować miesiącami
Progress 4GL / SlickGrid / Ant	Nieistotne dla celu		Nie inwestować dalej

Java

Twoja Java musi stać się **najmocniejszą częścią rozmowy**, ponieważ to właśnie na doświadczenie komercyjne w Javie będziesz się sprzedawał.

Aktualny świat Javy nie kończy się już na 21: Oracle wskazuje obecnie JDK 26 jako najnowsze wydanie, JDK 25 jako najnowsze LTS, a JDK 21 jako poprzednie LTS. ⁸ Rynek nie przeskakuje jednak natychmiast na każdą nową wersję: aktualne oferty nadal pokazują Java 17/21, Java 21+, a niektóre zaczynają już wymieniać 21/25 lub JDK 25. SmartBear chce Java 21+, Xebia Java 21/25, a część aktualnych ofert na Pracuj wymienia JDK 25 i Spring Boot 3.x/4.x. ⁹

Dla ciebie priorytetem jest więc:

Java 8 → 17 → 21 → świadomość 25, a nie zapamiętywanie release notes każdej wersji.

Musisz bardzo dobrze znać:

List, Set, Map, HashMap, HashSet, ArrayList, kontrakt equals/hashCode, Comparable vs Comparator, generics, wildcards ? extends / ? super, immutable objects, String, enumy, records, interfaces/default methods, abstract classes, wyjątki checked/unchecked, try-with-resources, streams, collectors, lambda, method references, Optional.

Na rozmowę powinieneś również rozumieć:

- heap vs stack,
- czym jest JVM, JIT i bytecode,
- podstawową ideę GC,
- strong/weak references na poziomie świadomości,
- race condition,
- thread safety,
- synchronized,
- volatile,
- atomics,

- `ExecutorService`,
- `Future` i `CompletableFuture`,
- concurrent collections.

Sabre w aktualnej juniorskiej ofercie na około dwa lata doświadczenia wprost wymienia **Java concurrency i profiling**, obok TDD, Linuxa, CI i REST. ¹⁰ Nie oznacza to, że masz stroić G1GC jak senior platform engineer. Masz potrafić odpowiedzieć, **dlaczego dwa wątki mogą zgubić aktualizację licznika i jak temu zapobiec**.

Spring

Tutaj masz obecnie największą dziurę.

I właśnie tutaj powinno pójść najwięcej czasu.

Spring Boot nie jest dodatkiem do Javy w tym segmencie rynku — w aktualnych ofertach modern Java backend pojawia się stale. SmartBear przy 1–2+ latach oczekuje Java 21+ i Spring Boot 3+, BAE minimum roku pracy z microservices/Spring Boot, Jacobs „extensive experience with Spring/Spring Boot”, a oferta OVOO przy minimum dwóch latach wymaga praktycznej znajomości Spring Boot i mikroserwisów. ¹¹

Obecna stabilna dokumentacja Springa pokazuje już linię Spring Boot 4.1. ¹² Nie oznacza to jednak, że masz uczyć się „Spring Boot 4” jako osobnego frameworka. Najważniejsze mechanizmy, które będą sprawdzane, są fundamentalne.

Musisz rozumieć:

Spring Core

- IoC,
- Dependency Injection,
- czym jest bean,
- component scanning,
- constructor injection,
- `@Component`, `@Service`, `@Repository`,
- `@Configuration`, `@Bean`,
- scopes,
- lifecycle na rozsądnym poziomie,
- dlaczego constructor injection jest preferowane.

Spring Boot

- `@SpringBootApplication`,
- starters,
- auto-configuration,
- `application.yml`,
- profiles,
- environment variables,
- `@ConfigurationProperties`,
- dependency management,
- Actuator na podstawowym poziomie.

Spring MVC

- `@RestController`,
- `@RequestMapping`,
- `@GetMapping`, `@PostMapping` itd.,
- path/query/body,
- request/response lifecycle na wysokim poziomie,
- serialization JSON,
- DTO,
- Bean Validation,
- global exception handling przez `@ControllerAdvice`.

Spring Data / Hibernate

- repositories,
- entities,
- persistence context,
- entity states,
- dirty checking,
- lazy/eager,
- relacje,
- cascade,
- `orphanRemoval`,
- JPQL,
- native query,
- pagination,
- N+1,
- `fetch join`,
- transactions.

Spring Security

- authentication vs authorization,
- filter chain na wysokim poziomie,
- password hashing,
- roles/authorities,
- JWT,
- stateless REST,
- co robi OAuth2 i OIDC.

Rockitworks przy około dwóch latach doświadczenia wymienia Spring Boot, Spring Security, JWT, `DataJpaTest`, `SpringBootTest`, JUnit i relacyjną bazę SQL. ¹³ To jest niemal gotowa lista tematów dla twojego projektu treningowego.

SQL, JPA i backend

SQL jest bardzo niedocenianym sposobem na odróżnienie „gościa, który zrobił tutorial Spring Boota” od kogoś wyglądającego jak developer z realnym doświadczeniem.

Jacobs wymaga SQL, SmartBear RDBMS, Rockitworks relacyjnej bazy, Kitopi wskazuje PostgreSQL, a aktualne oferty Pracuj wielokrotnie zestawiają Java/Spring z SQL, PostgreSQL, Hibernate czy innymi bazami. ¹⁴

Nie wystarczy:

```
SELECT * FROM users;
```

Powinieneś bez problemu pisać:

```
SELECT c.id,  
       c.name,  
       COUNT(o.id) AS order_count,  
       SUM(o.total_amount) AS total  
FROM customers c  
LEFT JOIN orders o ON o.customer_id = c.id  
WHERE o.created_at >= :from  
GROUP BY c.id, c.name  
HAVING COUNT(o.id) >= 3  
ORDER BY total DESC;
```

I umieć powiedzieć:

- czym różni się INNER JOIN od LEFT JOIN,
- WHERE od HAVING,
- PK od FK,
- po co jest indeks,
- dlaczego indeks przyspiesza read, ale kosztuje write/storage,
- co daje composite index i dlaczego kolejność kolumn ma znaczenie,
- co oznacza ACID,
- co może się stać przy dwóch równoległych transakcjach,
- isolation levels na poziomie praktycznym,
- optimistic vs pessimistic locking,
- kiedy ORM wywołuje SQL,
- czym jest N+1.

REST też potraktuj jako osobny skill.

Musisz umieć zaprojektować np.:

```
POST   /api/orders  
GET    /api/orders/{id}  
GET    /api/orders?page=0&size=20&status=PAID  
PATCH /api/orders/{id}/status  
DELETE /api/orders/{id}
```

i uzasadnić:

- 200 vs 201 vs 204 ,
- 400 vs 404 vs 409 ,
- idempotency,
- walidację,
- pagination,
- DTO vs entity,
- error response,
- versioning API.

Testy i narzędzia

Testing nie jest czymś „na później”. BAE wymienia pisanie testów i TDD, Jacobs JUnit, Rockitworks JUnit i springowe test slices, a OVOO jednostkowe i integracyjne. ¹⁵

Na twoim poziomie oczekiwałbym:

JUnit 5

- @Test ,
- assertions,
- parameterized tests,
- setup.

Mockito

- mock ,
- when ,
- verify ,
- argument matchers,
- kiedy mockować, a kiedy nie.

Spring tests

- @WebMvcTest ,
- MockMvc ,
- @DataJpaTest ,
- @SpringBootTest .

Integration

- PostgreSQL w Testcontainers,
- sprawdzanie prawdziwego repozytorium,
- test endpointu przechodzący przez większą część stacku.

Maven ma znacznie większy priorytet niż Gradle dla ciebie. Maven jest wymieniony w ofertach BAE, Jacobs i Kitopi, podczas gdy konkretne build toolsy zależą od projektu. ¹⁶

Umiej:

```
mvn clean
mvn test
mvn package
mvn verify
```

oraz czym są:

- dependency,
- plugin,
- lifecycle,
- scope,
- parent,
- BOM,
- profile.

Docker: musisz sam zrobić `Dockerfile` i `docker-compose.yml` dla aplikacji + PostgreSQL + np. Kafka.

Kubernetes: **nie musisz stawiać klastra produkcyjnego**. Aktualne juniorskie/okołojuniorskie oferty pokazują go jako znajomość/familiarity lub część stacku; BAE i Sabre umieszczają Docker/Kubernetes po stronie dodatkowych kompetencji, podczas gdy Jacobs oczekuje familiarności. ¹⁷

Czyli wystarczy wiedzieć:

Pod → Deployment → Service → ConfigMap → Secret

i rozumieć, dlaczego konteneryzacja to nie to samo co orkiestracja.

Kafka, mikroserwisy i cloud

Kafka jest **wartościowa, ale nie stawiaj jej przed Springiem, Hibernate czy SQL-em**.

W aktualnych ofertach pojawia się regularnie: Jacobs ma Kafka jako nice-to-have, BAE wskazuje ją wśród dodatkowych technologii, a aktualny Junior Java Developer na JJIT/Link Group jest tagowany Java + Spring + Kafka + Git. ¹⁸ Jednocześnie wiele bardziej rozbudowanych ofert z Kafką dotyczy mida/seniora, więc nie traktowałbym jej jako absolutnej bariery wejścia na twoim poziomie. ¹⁹

Masz rozumieć:

- broker,
- topic,
- partition,
- offset,
- producer,
- consumer,
- consumer group,
- ordering w obrębie partycji,
- at-most-once / at-least-once,
- duplikaty,

- idempotent consumer,
- retry,
- dead letter topic,
- dlaczego event-driven $\neq$ REST.

Microservices podobnie.

Nie ucz się pięćdziesięciu patternów mikroserwisowych.

Umiej odpowiedzieć:

Dlaczego nie wszystko powinno być mikroserwisem?

Co jest trudniejsze w systemie rozproszonym niż w monolicie?

Jak obsłużyć częściową awarię?

Synchroniczny REST czy komunikacja asynchroniczna?

Co z transakcją obejmującą kilka usług?

Cloud też nie powinien pożyć twojego planu. Sabre wymienia GCP, Aviva AWS jako nice-to-have, aktualne oferty często zawierają AWS lub inny cloud, ale dla kandydatów z około dwóch lat doświadczenia występuje zarówno jako requirement, jak i kompetencja dodatkowa. ²⁰

Wybierz jedno — najlepiej AWS — i poznaj tylko:

EC2/ECS na poziomie pojęcia, S3, RDS, IAM, CloudWatch, secrets/configuration, podstawę VPC.

Na razie żadnego miesięcznego kursu „AWS Solutions Architect 50 hours”.

React: brutalna odpowiedź

Na twoim miejscu odłożyłbym poważną naukę Reacta.

Nie porzucaj go całkowicie. Ale masz znacznie bardziej palący problem.

React/Angular rzeczywiście poszerzają pulę ofert, bo istnieje sporo Java Full Stack. Aktualne wyniki Pracuj pokazują role łączące Java/Spring z Angular/React, a na JJIT pojawiają się juniorskie Java + frontend. ²¹ Jednocześnie w backendowej ofercie Rockitworks HTML/CSS/Node/React/Angular znajdują się w sekcji **nice to have**, podczas gdy Java, Spring Boot, Spring Security, JWT, SQL i testy są bazowym wymaganiem. ¹³

Dlatego przy twoich ograniczonych godzinach:

80-90% czasu technicznego → backend.

React:

- JSX,

- komponent,
- props/state,
- hooks,
- request do REST,
- formularz,

i wystarczy.

Nie Redux, Next.js internals, React Query internals, frontend architecture i pięć bibliotek CSS.

Co prawdopodobnie zobaczysz na rekrutacji

Trzeba odróżnić trzy rzeczy.

Warstwa	Co się dzieje
Ogłoszenie HR	często szeroka lista Java + Spring + Kafka + Kubernetes + AWS + Docker + CI/CD + mikroserwisy
Rzeczywista codzienna robota	zwykle tylko część tego stacku i dużo istniejącego kodu
Rozmowa techniczna	fundamenty Java + Spring + DB/JPA + REST + testy + rozumowanie

To ostatnie jest moim wnioskiem z powtarzających się wymagań w ofertach i typowych formatów assessmentów, a nie uniwersalną regułą każdej firmy. Globalne dane HackerRank pokazują, że mimo niezadowolenia kandydatów zadania algorytmiczne nadal funkcjonują; jednocześnie 66% badanych developerów deklarowało preferowanie praktycznych zadań codingowych zamiast testów oderwanych od rzeczywistej pracy. ²²

Najczęstsze tematy techniczne — trzydzieści pozycji

W kolejności, w jakiej **ja** uczyłbym się ich pod twój profil:

1. Collections: List, Set, Map.
2. `HashMap` i kontrakt `equals/hashCode`.
3. OOP: inheritance vs composition, encapsulation, polymorphism.
4. Generics.
5. Streams.
6. Lambdy i functional interfaces.
7. `Optional`.
8. Exceptions.
9. Immutability, records, enum.
10. SOLID i clean code.
11. Thread safety i podstawy concurrency.
12. JVM: heap, stack, GC, JIT.
13. Java 8 vs 11 vs 17 vs 21/25.
14. Dependency Injection / IoC.
15. Spring bean i application context.
16. Spring Boot auto-configuration.
17. Spring MVC i request lifecycle.

18. REST/HTTP/status codes.
19. DTO, validation, exception handling.
20. JPA/Hibernate persistence context.
21. Lazy/eager + N+1.
22. Relacje JPA i cascade.
23. `@Transactional` i transakcje.
24. SQL JOIN/GROUP BY/index.
25. Isolation levels i locking.
26. JUnit + Mockito.
27. Unit vs integration tests.
28. Maven/Git/Docker/CI.
29. Microservices + Kafka fundamentals.
30. Spring Security/JWT/OAuth2 fundamentals.

Czy potrzebujesz LeetCode?

Tak, ale nie rób z tego religii.

W zwykłym Java/Spring software house/enterprise/backend nie widać w ogłoszeniach obsesji na punkcie algorytmów porównywalnej z wymaganiami dotyczącymi Springa, SQL, REST czy testów. Jednocześnie assessmenty algorytmiczne nadal istnieją, a globalne dane HackerRank potwierdzają, że tego typu testy nie zniknęły. ²²

Dla ciebie:

około 30-50 dobrze przerobionych problemów > 200 odklepanych z pamięci.

Poznaj:

- array/string,
- HashMap/HashSet,
- two pointers,
- sliding window,
- stack/queue,
- binary search,
- sorting,
- podstawowe recursion,
- BFS/DFS jako świadomość,
- Big-O.

Powinieneś być w stanie zrobić Easy bez paniki i rozsądne Medium po zastanowieniu.

Dopiero jeżeli będziesz celował w Google-like, trading, mocny fintech albo firmy słynące z DSA, zwiększ ten zakres.

AI na rozmowach i w pracy

Najważniejsze: **korzystanie z AI samo w sobie przestało być czymś, co trzeba ukrywać.**

W Stack Overflow Developer Survey 2025 84% respondentów korzystało lub planowało korzystać z narzędzi AI w procesie developmentu, a 51% profesjonalnych developerów deklarowało codzienne

użycie. ²³ W badaniu HackerRank 97% respondentów korzystało z AI, a niemal jedna trzecia kodu w ich badanej populacji była deklarowana jako generowana z pomocą AI; raport równocześnie wskazuje, że firmy nadal ustalają zasady pozwalania na AI podczas assessments. ²²

Co ciekawsze z perspektywy Polski, to nie jest tylko globalny hype. SmartBear w aktualnym juniorskim ogłoszeniu 1–2+ YOE wymienia **experience with AI for accelerating development activities**, a Kitopi traktuje doświadczenie z Copilot/Cursor jako plus dla Junior Software Engineer. ²⁴ Xebia idzie dalej: w aktualnej ofercie Mid jako must-have wymienia AI Tools, Claude Code, GitHub Copilot i Cursor oraz praktyczne stosowanie asystentów AI w SDLC. ⁴

Na forach również pojawia się nowy typ rozmowy, podczas której AI jest dozwolone, a dyskusje kandydatów i interviewerów koncentrują się na tym, czy kandydat potrafi **zweryfikować wynik, zakwestionować błędną sugestię i wyjaśnić własne decyzje**. To jednak traktuję wyłącznie jako anegdotyczny sygnał z community, a nie twardą statystykę rynku. ²⁵

I tutaj dochodzimy do twojego największego osobistego ryzyka.

Nie to, że używasz AI.

Tylko:

„koduję z AI tak szybko, że później nie wiem dokładnie, dlaczego ten kod działa”.

To trzeba naprawić.

Nie przez wyrzucenie AI.

Przez wprowadzenie dwóch trybów:

Tryb produkcyjny: AI dozwolone.

Tryb treningowy: AI wyłączone.

Rzeczy, które powinienś napisać bez AI

Przed rozpoczęciem rekrutacji powinienś umieć z pustego pliku zrobić przynajmniej:

1. Klasy domenowe z sensownym OOP.
2. `equals/hashCode`.
3. Sortowanie z `Comparator`.
4. Transformacje kolekcji.
5. Stream z `filter/map/groupingBy`.
6. Kod wykorzystujący generics.
7. Obsługę wyjątków.
8. Prostą operację thread-safe.
9. Spring `Controller → Service → Repository`.
10. CRUD REST.
11. Request/response DTO.
12. Bean Validation.
13. `@ControllerAdvice`.

14. Relację JPA `OneToMany/ManyToOne`.
15. Pageable endpoint.
16. Zapytanie SQL z kilkoma JOIN-ami i agregacją.
17. Unit test JUnit + Mockito.
18. Test integracyjny endpointu/repository.
19. Dockerfile + prosty Compose.
20. Prosty Kafka producer/consumer po wcześniejszym przećwiczeniu.

Nie chodzi o wykuwanie anotacji na pamięć.

Chodzi o to, żebyś nie siedział sparaliżowany, kiedy interviewer powie:

„Mamy użytkownika i zamówienia. Napisz endpoint zwracający zamówienia użytkownika i dodaj test.”

Przykładowe live codingi

Najbardziej użyteczne treningi dla ciebie to:

1. Napisanie małego REST endpointu.
2. Dodanie walidacji i obsługi błędu.
3. Napisanie service method oraz unit testu.
4. Naprawienie testu z Mockito.
5. Refaktor dużej metody z `if/else`.
6. Zamiana pętli na sensowny Stream — albo uzasadnienie, dlaczego pętla jest czytelniejsza.
7. Grupowanie obiektów po kluczu.
8. Znalezienie duplikatów w kolekcji.
9. Implementacja prostego cache.
10. Napisanie SQL JOIN.
11. Naprawienie N+1.
12. Zaprojektowanie endpointów CRUD.
13. Code review kiepskiego kodu.
14. Wykrycie race condition.
15. Parsowanie danych / transformacja kolekcji.
16. Dodanie paginacji.
17. Dodanie transakcji.
18. Implementacja optimistic locking.
19. Napisanie konsumenta eventu.
20. Przeanalizowanie kodu wygenerowanego przez AI i znalezienie błędu.

Konkretna checklista i pytania do powtórki

MUST HAVE

Nie szedłbym świadomie na rozmowy na Java backend, dopóki większość tego nie jest zielona:

Java

- [] OOP
- [] collections
- [] HashMap

- ☐ equals/hashCode
- ☐ generics
- ☐ streams
- ☐ lambdas
- ☐ Optional
- ☐ exceptions
- ☐ immutability
- ☐ records
- ☐ Java 8/17/21 najważniejsze różnice
- ☐ concurrency fundamentals
- ☐ JVM fundamentals

Spring

- ☐ IoC/DI
- ☐ Spring beans
- ☐ Spring Boot
- ☐ REST
- ☐ Spring MVC
- ☐ validation
- ☐ exception handling
- ☐ config/profiles
- ☐ Spring Data JPA
- ☐ transactions

Database

- ☐ SELECT
- ☐ JOIN
- ☐ GROUP BY
- ☐ HAVING
- ☐ indexes
- ☐ transactions
- ☐ isolation
- ☐ JPA relationships
- ☐ lazy/eager
- ☐ N+1

Testing

- ☐ JUnit 5
- ☐ Mockito
- ☐ unit test
- ☐ integration test
- ☐ MockMvc lub odpowiednik

Development

- ☐ Git
- ☐ Maven
- ☐ REST/HTTP
- ☐ JSON

- ☐ Docker fundamentals

Ta koncentracja odpowiada powtarzającemu się rdzeniowi aktualnych ofert dla Java Junior/Junior+/early Mid: Java, Spring/Spring Boot, REST, relacyjne DB/SQL, testing, Git oraz narzędzia build/CI. 26

BARDZO WAŻNE

- ☐ Spring Security
- ☐ JWT
- ☐ OAuth2 fundamentals
- ☐ Testcontainers
- ☐ Docker Compose
- ☐ CI/CD
- ☐ Linux
- ☐ SOLID
- ☐ design patterns
- ☐ microservices fundamentals
- ☐ Kafka fundamentals
- ☐ observability/logging
- ☐ optimistic/pessimistic locking
- ☐ API pagination/filtering
- ☐ Flyway/Liquibase

CI/CD i konteneryzacja pojawiają się już nawet przy ofertach blisko twojego poziomu: BAE chce znajomości CI, Jacobs Docker/Kubernetes, Aviva CI/CD, a OVOO Docker, Kubernetes i CI/CD. 27

NICE TO HAVE

- ☐ Kubernetes basics
- ☐ AWS basics
- ☐ Redis
- ☐ RabbitMQ
- ☐ WireMock
- ☐ REST Assured
- ☐ OpenAPI/Swagger
- ☐ metrics/Micrometer
- ☐ Prometheus/Grafana
- ☐ NoSQL basics
- ☐ React basics
- ☐ Angular awareness
- ☐ Gradle basics
- ☐ Spring Cloud awareness

NIE TRACIĆ TERAZ CZASU

Nie znaczy „bezużyteczne”. Znaczy „fatalny ROI dla ciebie w najbliższych sześciu miesiącach”.

Nie wchodziłbym głęboko w:

- Kubernetes administration,
- Terraform,
- service mesh,

- Istio,
- zaawansowane AWS,
- certyfikaty cloud,
- zaawansowane GC tuning,
- lock-free programming,
- Spring WebFlux,
- Reactor internals,
- GraphQL,
- CQRS/Event Sourcing od strony akademickiej,
- kilkadziesiąt wzorców DDD,
- Spring Cloud w całej szerokości,
- zaawansowany React,
- Redux ecosystem,
- frontend architecture,
- Progress 4GL,
- SlickGrid,
- Ant ponad to, czego wymaga obecna praca.

I absolutnie nie poświęcałbym kolejnych kilkadziesiątu godzin na bycie coraz lepszym w twoim obecnym legacy stacku, jeżeli firma sama cię do tego nie zmusza.

Bank pytań rekrutacyjnych

Poniższe pytania nie są kopiowane z jednej firmy; to zestaw treningowy zbudowany wokół tematów powtarzających się w wymaganiach ofert Java/Spring oraz typowych obszarów assessments. Aktualne oferty dla twojego przedziału doświadczenia potwierdzają nacisk na Java/OOP, Spring, REST, SQL/RDBMS, testing, design patterns, CI/CD i narzędzia developerskie. 28

Java

1. Jaka jest różnica między `==` a `equals()`?
2. Dlaczego przy nadpisaniu `equals()` trzeba nadpisać `hashCode()`?
3. Jak w uproszczeniu działa `HashMap`?
4. `ArrayList` vs `LinkedList` — kiedy czego użyć?
5. `HashSet` vs `TreeSet`.
6. Co daje immutability?
7. Jaka jest różnica między checked i unchecked exception?
8. Co oznacza `final` dla zmiennej, metody i klasy?
9. Interface vs abstract class.
10. Co to jest generic type i po co go używamy?
11. Co oznaczają `? extends T` i `? super T`?
12. Czym jest functional interface?
13. Jak działa Stream i czym różni się od kolekcji?
14. `map()` vs `flatMap()`.
15. Do czego służy `Optional` i jak go nie używać?
16. Co dają records?
17. Co może spowodować race condition?
18. `synchronized` vs `volatile`.
19. Co robi `ExecutorService`?
20. Heap vs stack.

Spring

1. Co to jest IoC?
2. Co to jest Dependency Injection?
3. Co jest beanem Springa?
4. `@Component` vs `@Service` vs `@Repository`.
5. Dlaczego constructor injection?
6. Co robi `@SpringBootApplication`?
7. Na czym polega auto-configuration?
8. `@RestController` vs `@Controller`.
9. Jak zrobićbyś globalną obsługę wyjątków?
10. Jak zwalidujesz request?
11. Jak konfigurujesz różne środowiska?
12. Co robi `@Transactional`?
13. Dlaczego `@Transactional` może nie zadziałać przy wywołaniu metody z tej samej klasy?
14. Czym jest Spring Security filter chain?
15. Authentication vs authorization.

JPA / SQL

1. Co to jest persistence context?
2. Lazy vs eager loading.
3. Czym jest N+1 i jak go znaleźć?
4. `OneToMany` vs `ManyToOne`.
5. Co robi cascade?
6. Co robi `orphanRemoval`?
7. `INNER JOIN` vs `LEFT JOIN`.
8. `WHERE` vs `HAVING`.
9. Jak działa indeks?
10. Co to jest transaction isolation?
11. Optimistic vs pessimistic locking.

Backend / engineering

1. `PUT` vs `PATCH`.
2. Kiedy zwrócisz 400, 404, 409, 500?
3. Unit test vs integration test.
4. Co zyskujemy i co tracimy przechodząc z monolitu na mikroserwisy?

Jeżeli na co najmniej 40 z tych 50 pytań potrafisz odpowiedzieć **własnymi słowami, podać przykład i dopytany wejść poziom głębiej**, to twoja sytuacja rekrutacyjna będzie nieporównywalnie lepsza niż obecnie.

Projekt, który powinien zastąpić brak komercyjnego Springa

Nie rób kolejnego:

Todo API: User, Task, CRUD.

Potrzebujesz jednego projektu, który pozwoli ci podczas rozmowy udowodnić:

„Nie używałem tego komercyjnie, ale naprawdę potrafię na tym pracować.”

Nazwijmy go roboczo **OrderFlow**.

Domena

System zamówień dla sklepu/marketplace.

Mamy:

```
User
Product
Inventory
Order
OrderItem
Payment
```

Workflow:

```
User
  ↓
POST /orders
  ↓
OrderService
  ↓
sprawdzenie inventory
  ↓
rezerwacja produktu
  ↓
zapis Order
  ↓
publikacja OrderCreated
  ↓
Kafka
  ↓
Payment/Notification consumer
```

To jest dużo lepsze niż rozdmuchane cztery mikroserwisy po 150 linii każdy.

Stack projektu

Zrobiłbym:

```
Java 21
Spring Boot
Spring MVC
Spring Data JPA
Hibernate
```

```
PostgreSQL
Flyway
Spring Security
JWT / OAuth2 Resource Server
Bean Validation
JUnit 5
Mockito
MockMvc
Testcontainers
Kafka
Docker
Docker Compose
Maven
GitHub Actions
OpenAPI
Spring Boot Actuator
```

Java 21 jest nadal bardzo sensowną bazą portfolio, mimo że Java 25 jest obecnym LTS, ponieważ współczesne oferty pokazują zarówno Java 21+, jak i 21/25 oraz stopniowe przechodzenie na 25. ²⁹ Projekt warto później uruchomić również na nowszym JDK i wiedzieć, jakie wersje wykorzystuje rynek.

Funkcjonalności

Zrób:

CRUD produktów, ale potem:

- paginacja,
- filtrowanie,
- sortowanie,
- rejestracja/logowanie,
- role USER/ADMIN,
- składanie zamówienia,
- transakcja,
- inventory,
- optimistic locking,
- walidacja,
- sensowne status codes,
- global exception handler,
- Kafka event `OrderCreated`,
- consumer,
- idempotency,
- testy.

Dodaj przypadek równoległy:

```
w magazynie została 1 sztuka
```

User A kupuje
User B kupuje jednocześnie

I rozwiąż problem.

To jest świetny punkt wyjścia do rozmowy o:

- concurrency,
- transactions,
- optimistic locking,
- consistency,
- JPA,
- HTTP conflicts.

Architektura

Nie rób od razu:

```
user-service
order-service
inventory-service
product-service
payment-service
notification-service
api-gateway
service-discovery
config-server
```

To będzie tutorialowa imitacja mikroserwisów.

Zrób **porządny modularny monolit**, np.:

```
order
├─ api
├─ application
├─ domain
└─ infrastructure

inventory
├─ api
├─ application
├─ domain
└─ infrastructure
```

i jeden sensowny async integration point przez Kafka.

Potem możesz powiedzieć na rozmowie:

„Świadomie nie rozdzieliłem wszystkiego na mikroserwisy, bo w tym zakresie projektu koszt operacyjny i distributed-system complexity byłyby większe od korzyści.”

Taka odpowiedź robi na mnie znacznie lepsze wrażenie niż:

„Mikroserwisy są nowoczesne, więc zrobiłem ich osiem.”

Testy

Powinieneś mieć:

```
unit
└─ OrderServiceTest

web slice
└─ OrderControllerTest

repository
└─ OrderRepositoryTest

integration
└─ OrderFlowIntegrationTest
```

PostgreSQL w Testcontainers.

Nie H2 udające PostgreSQL.

CI

GitHub Actions:

```
checkout
↓
setup Java
↓
mvn verify
↓
tests
↓
build
```

Opcjonalnie obraz Dockera.

Dzięki temu w jednym projekcie pokryjesz większość luki pomiędzy twoim obecnym doświadczeniem a rdzeniem współczesnych ogłoszeń: Spring Boot, REST, SQL/RDBMS, testowanie, build, kontenery, CI oraz dodatkowo messaging. Ten zestaw dobrze odpowiada wymaganiom powtarzającym się w aktualnych ofertach dla 1–2-letnich developerów. ²⁸

Plan przygotowania na sześć miesięcy

Przy 6–8 godzinach tygodniowo masz przez 26 tygodni około **156–208 godzin**. To absolutnie wystarczy na naprawienie twojej sytuacji, ale **nie wystarczy na naukę wszystkiego**, więc priorytety są kluczowe.

Nie rób:

- 2 tygodnie kursu Java
- 2 tygodnie kursu Spring
- 2 tygodnie kursu SQL
- 2 tygodnie kursu Docker
- ...

Po trzech miesiącach będziesz miał 14 skończonych tutoriali i zero samodzielności.

Rób jeden projekt i dokładaj do niego wiedzę.

Pierwszy miesiąc — odzyskanie kontroli nad Javą

Cel: przestać być zależnym od AI przy podstawowym kodowaniu.

Okolo 7 h/tydzień:

Czas	Zadanie
2 h	Core Java
2 h	ćwiczenia bez AI
1,5 h	SQL
1,5 h	początki projektu

Tydzień po tygodniu:

Pierwsza część

- collections,
- equals/hashCode,
- OOP,
- exceptions.

Druga

- generics,
- lambdas,
- Stream.

Trzecia

- Optional,

- immutability,
- records,
- Java 17/21.

Czwarta

- JVM,
- concurrency fundamentals,
- powtórka.

Każdego tygodnia minimum godzina kodowania **AI OFF**.

Drugi miesiąc — Spring od fundamentów

Cel:

```
HTTP
↓
Controller
↓
Service
↓
Repository
↓
PostgreSQL
```

Tematy:

- IoC/DI,
- beans,
- Spring Boot,
- configuration,
- MVC,
- REST,
- DTO,
- validation,
- error handling,
- OpenAPI.

Po miesiącu masz z pustego repo postawić backend samodzielnie.

Trzeci miesiąc — SQL + Hibernate + transactions + testing

Tu robisz największy skok jakościowy.

Tematy:

- PostgreSQL,
- JOIN,
- indexes,

- transactions,
- JPA relationships,
- persistence context,
- lazy/eager,
- N+1,
- pagination,
- locking,
- `@Transactional`,
- JUnit,
- Mockito,
- MockMvc,
- Testcontainers.

Na koniec miesiąca masz umieć dostać zepsute API i znaleźć:

dla czego robi 101 SQL queries zamiast dwóch.

Czwarty miesiąc — production-like backend

Dodajesz:

- Spring Security,
- JWT,
- Docker,
- Docker Compose,
- Flyway,
- logging,
- Actuator,
- GitHub Actions,
- Linux fundamentals.

W tym momencie projekt powinien już wyglądać jak coś, co można pokazać rekruterowi.

I od tego miesiąca zacząłbym pierwsze próbne rekrutacje.

Nie czekałbym, aż „będę gotowy”.

Bo prawdziwa rozmowa pokaże braki szybciej niż kolejny kurs.

Piąty miesiąc — mikroserwisy, messaging i cloud

Dopiero teraz:

- Kafka,
- async communication,
- event-driven,
- retries,
- idempotency,
- microservices trade-offs,
- Docker deeper,
- Kubernetes basics,

- AWS fundamentals.

Nie próbuj zostać DevOpsem.

Masz umieć rozmawiać z DevOpsem i rozumieć, jak twoja aplikacja jest budowana oraz uruchamiana.

Szósty miesiąc — tryb rekrutacyjny

Tutaj ograniczasz nowe technologie.

Tydzień:

```
2 h – Core Java / Spring interview questions
2 h – live coding bez AI
1 h – SQL
1 h – projekt / poprawki
1 h – CV / aplikacje / rozmowy
```

Co kilka dni:

symulacja 60–90 minut.

Bez Google.

Bez ChatGPT.

Bez Copilota.

Przykładowo:

Napisz REST API dla rezerwacji miejsc.

Potem sam sobie zadaj pytania:

- dlaczego takie DTO?
- co z concurrency?
- co z transakcją?
- co z błędem?
- co z testem?
- co z N+1?
- co jeśli dwa requesty przyjdą naraz?

To właśnie będzie antidotum na twój obecny sposób pracy z AI.

Co bym zrobił na twoim miejscu

Najpierw bardzo konkretnie:

nie panikowałbym przez Progress 4GL, SlickGrid czy starą Java EE.

To, że pierwsze półtora roku kariery trafiłeś do legacy, nie przekreśla cię jako Java Developera.

Ale też nie będę ci wciskał, że „wszystko jest super”.

Jeżeli przez następny rok nadal będziesz tylko klepał legacy z AI i nie zbudujesz mocnego fundamentu modern Java backend, to z czasem problem będzie coraz większy.

Rynek dla czystych juniorów jest mały — na Just Join IT junior stanowił mniej niż 5% ofert w danych za 2025 — podczas gdy grupa mid odpowiadała za prawie 44%. ³⁰ A wymagania przy ofertach 1–2 YOE już dziś potrafią obejmować Spring Boot, REST, SQL, testowanie i czasem elementy CI/CD, kontenerów czy AI-assisted development. ³¹

Czyli masz właśnie dobre okno, żeby się przestawić.

Jak podzieliłbym czas

Przy 7 godzinach tygodniowo:

Obszar	Udział
Core Java	20%
Spring Boot / MVC	25%
SQL + JPA/Hibernate	15%
Testing	10%
Projekt	15%
Docker/CI/Git/Linux	5%
Kafka/microservices/cloud	5%
Algorytmy/interview drills	5%

React dostaje praktycznie **zero dodatkowego priorytetu przez pierwsze 3–4 miesiące**.

Mój ranking tematów dla ciebie

Gdybym miał ci zabrać wszystkie technologie i pozwolić zachować tylko kilka, kolejność byłaby taka:

Java → Spring Boot → REST → SQL → JPA/Hibernate → testing → Git/Maven → Docker → transactions → Spring Security → Kafka → CI/CD → cloud/Kubernetes → React.

Nie odwrotnie.

Najważniejsze „osiemdziesiąt/dwadzieścia”

Gdybym był dokładnie w twojej sytuacji i miał tylko pół roku, moje 20% tematów generujące 80% wyniku wyglądałoby tak:

Core Java

Musisz być niebezpiecznie dobry w podstawach:

```
collections
OOP
equals/hashCode
generics
streams
exceptions
concurrency basics
JVM basics
```

Spring Boot

Musisz samodzielnie potrafić przejść:

```
request
↓
controller
↓
service
↓
transaction
↓
repository
↓
Hibernate
↓
PostgreSQL
↓
response
```

i wyjaśnić **co się dzieje na każdym etapie**.

SQL + Hibernate

Musisz rozumieć, że:

```
order.getItems()
```

to czasami nie jest zwykłe odczytanie pola, tylko potencjalny query do DB.

I że:

```
orders.stream()
    .map(Order::getCustomer)
```

może doprowadzić do lawiny zapytań.

Testing

Musisz potrafić napisać test bez Copilota.

REST/HTTP

Musisz umieć zaprojektować normalne API, a nie tylko znać `@GetMapping`.

Projekt

Jeden.

Ale dobry.

Nie pięć CRUD-ów.

AI-off coding

Minimum jedna sesja tygodniowo.

To szczególnie ważne dla ciebie.

Czego nie próbowałbym nadrobić

Nie próbowałbym zostać jednocześnie:

```
Java developer
+
React developer
+
AWS developer
+
Kubernetes engineer
+
Kafka expert
+
system designer
```

To jest najszybsza droga do bycia słabym we wszystkim.

Aktualne oferty 1–2 YOE pokazują, że rdzeniem nadal jest mocna Java plus Spring/REST/RDBMS i podstawowe praktyki software-engineeringowe; bardziej infrastrukturalne technologie często są rozszerzeniem tego rdzenia. ²⁸

Jak wykorzystać obecne legacy zamiast się go wstydzić

W CV nie pisz tylko:

```
Java EE
Progress 4GL
SlickGrid
Ant
```

bo to rzeczywiście wygląda źle.

Opisuj **problemy i odpowiedzialność**:

Development and maintenance of enterprise Java applications.

Implementing business functionality in an existing production system.

Debugging and resolving defects across backend and frontend layers.

Working with relational data and complex business workflows.

Refactoring and extending legacy code while maintaining backward compatibility.

Cooperating with analysts/testers/business stakeholders.

Investigating production and integration issues.

A potem osobna sekcja:

Modern backend project / technical development:

```
Java 21
Spring Boot
Spring MVC
Spring Data JPA
Hibernate
PostgreSQL
JUnit 5
Mockito
Testcontainers
Kafka
Docker
GitHub Actions
```

To zmienia narrację z:

„biedny junior utknął w starościach”

na:

„developer z doświadczeniem produkcyjnym, który zna legacy i świadomie przeszedł na nowoczesny stack”.

I to jest właśnie profil, który chcesz sprzedać.

Ostateczna odpowiedź na pytanie, na co przeznaczyć 80% czasu

Gdybym miał na twoim miejscu tylko sześć miesięcy:

nie uczyłbym się szerzej. Uczyłbym się głębiej.

Doprowadziłbym do tego, żebyś dostał na rozmowie:

```
Java
Spring
SQL
JPA
REST
JUnit
```

i pomyślał:

„zajebicie, pytajcie”.

A nie:

„mam nadzieję, że nie zapytają, bo normalnie Claude mi to pisze”.

Twój idealny punkt po sześciu miesiącach nie wygląda tak:

„znam 25 technologii”.

Tylko:

„umiem samodzielnie zbudować, przetestować, uruchomić i wytłumaczyć normalny backend w Javie i Spring Boot”.

Rynek pokazuje dokładnie w tę stronę: nawet obecne oferty dla około 1–2 lat doświadczenia łączą Java z nowoczesnym Spring Boot, REST/RDBMS, testowaniem i praktykami produkcyjnymi, natomiast React, Kafka, Kubernetes czy cloud częściej rozszerzają ten zestaw, niż go zastępują. ²⁶

To jest twój najkrótszy most ze starej Java EE do współczesnego Java/Spring backendu.

- 2 [REPORT] IT Career in Poland 2026. Guide to the Job Market – Branża IT w Polsce - No Fluff Insights
<https://nofluffjobs.com/en/insights/it-career-poland-2026/>
- 3 Aviva zatrudnia na stanowisko MID Software Engineer (Java) w Warszawa | LinkedIn
<https://pl.linkedin.com/jobs/view/mid-software-engineer-java-at-aviva-4317422128>
- 4 Middle Java Developer Job | Backend | Xebia | Warszawa | No Fluff Jobs.
<https://nofluffjobs.com/job/middle-java-developer-xebia-warszawa-1>
- 5 7 9 11 24 Junior Backend Engineer - Swagger Portal - SmartBear
<https://justjoin.it/job-offer/smartbear-junior-backend-engineer---swagger-portal-wroclaw-java>
- 6 14 Junior/Mid Java Developer w firmie Jacobs — Kraków | Oferty pracy na LinkedIn
<https://pl.linkedin.com/jobs/view/junior-mid-java-developer-at-jacobs-4384324029>
- 8 29 Java Downloads | Oracle
<https://www.oracle.com/java/technologies/downloads/>
- 10 20 Java Software Engineer I (Junior) w firmie Sabre Poland — Kraków | Oferty pracy na LinkedIn
<https://pl.linkedin.com/jobs/view/java-software-engineer-i-junior-at-sabre-poland-4123361145>
- 12 Dependency Versions :: Spring Boot
https://docs.spring.io/?utm_source=chatgpt.com
- 13 Java Developer w firmie Rockitworks — Warszawa | Oferty pracy na LinkedIn
<https://pl.linkedin.com/jobs/view/java-developer-at-rockitworks-4311575398>
- 15 16 17 18 26 27 28 31 Junior Java Developer w firmie BAE Systems Digital Intelligence — Poznań | Oferty pracy na LinkedIn
<https://pl.linkedin.com/jobs/view/junior-java-developer-at-bae-systems-digital-intelligence-4345471855>
- 19 21 Praca java – it.Pracuj.pl
https://it.pracuj.pl/praca/java%3Bkw?utm_source=chatgpt.com
- 22 2025 Developer Skills Report
<https://embed.hackerrank.com/reports/developer-skills-report-2025>
- 23 2025 Stack Overflow Developer Survey
https://survey.stackoverflow.co/?utm_source=chatgpt.com
- 25 What are interviewers actually looking for in a "use an AI ...
https://www.reddit.com/r/ExperiencedDevs/comments/1vivvyh/what_are_interviewers_actually_looking_for_in_a/?utm_source=chatgpt.com